

Understanding Python Metaprogramming

Solutions to Metaprogramming Exercises

Exercise 1: Dynamic Metadata Introspection - Solution

Objective: Access dunder attributes like `__name__`, `__bases__`, `__mro__`, and `__dict__` to audit class structure dynamically.

Code Solution:

```
def inspect_class(cls):
    name = cls.__name__
    bases = cls.__bases__
    mro = cls.__mro__
    attributes = [
        attr for attr in cls.__dict__
        if not attr.startswith("__")
    ]
    return {
        "name": name,
        "bases": bases,
        "mro": mro,
        "attributes": attributes
    }
```

Explanation:

This exercise reviews dynamic introspection. Accessing `cls.__name__`, `cls.__bases__`, and `cls.__mro__` retrieves essential runtime hierarchies. Filtering `cls.__dict__` with a simple list comprehension screens out standard dunder namespace attributes.

Exercise 2: Dynamic Configuration Wrapper - Solution

Objective: Turn a dictionary data source into an attribute-based interface using `__getattr__`.

Code Solution:

```
class ReadOnlyConfig:
    def __init__(self, data):
        object.__setattr__(self, "_data", dict(data))

    def __getattr__(self, name):
        if name not in self._data:
            raise AttributeError(f"Attribute '{name}' not found")
        value = self._data[name]
        if isinstance(value, dict):
            return ReadOnlyConfig(value)
        return value

    def __setattr__(self, name, value):
        raise AttributeError("ReadOnlyConfig is immutable.")
```

Explanation:

Overriding `__getattr__` intercepts failed attribute lookups and redirects them into the internal `_data` dictionary. If the retrieved value is itself a dictionary, it is recursively wrapped in a `ReadOnlyConfig` instance. The custom `__setattr__` guards against mutation.

Exercise 3: Strict Attribute Validation - Solution

Objective: Intercept attribute assignment to validate or normalise values.

Code Solution:

```
class StrictEmployee:
    def __setattr__(self, name, value):
        if name == "name":
            if not isinstance(value, str):
                raise TypeError("Employee name must be of type 'str'")
            if len(value.strip()) == 0:
                raise ValueError("Employee name cannot be empty")
        elif name == "salary":
            if not isinstance(value, (int, float)):
                raise TypeError("Salary must be a numeric value")
            if value < 0:
                raise ValueError("Salary cannot be negative")
        object.__setattr__(self, name, value)
```

Explanation:

Overriding `__setattr__` intercepts attribute writes. To avoid infinite recursion, the actual write is delegated to `object.__setattr__(self, name, value)`. Type and value checks run before the write is committed.

Exercise 4: Generating a Class with `type()` - Solution

Objective: Dynamically create custom class architectures programmatically.

Code Solution:

```
def class_factory(class_name, fields):
    def __init__(self, *args):
        if len(args) != len(fields):
            raise TypeError("Argument count mismatch")
        for field, value in zip(fields, args):
            setattr(self, field, value)

    namespace = {"__init__": __init__}
    return type(class_name, (), namespace)
```

Explanation:

This demonstrates programmatic class creation. A custom `__init__` function binds arguments to instance fields, is placed in a namespace dictionary, and is passed to `type(name, bases, namespace)`

to produce a new, dynamic class.

Exercise 5: Dynamic API Method Generator - Solution

Objective: Attach dynamic helper methods to classes programmatically based on a configuration list of strings.

Code Solution:

```
def generate_api_client(endpoints):
    namespace = {}
    for endpoint in endpoints:
        method_name = f"get_{endpoint}"

        def make_method(ep=endpoint):
            return lambda self: self.mock_get(ep)

        namespace[method_name] = make_method()
    return type("DynamicApiClient", (ApiClient,), namespace)
```

Explanation:

To bind dynamic methods safely, lambdas are defined using a default-parameter capture trick (`ep=endpoint`) inside a helper function (`make_method`). This prevents late-binding bugs where every generated method would otherwise resolve to the last item in the list.

Exercise 6: Metaclass Interface Verifier - Solution

Objective: Enforce structural layout rules and docstring presence at class compilation time.

Code Solution:

```
class InterfaceVerifier(type):
    def __new__(mcls, name, bases, namespace):
        is_base = name in ("BaseComponent", "Base") or not bases
        if not is_base:
            doc = namespace.get("__doc__")
            if not doc or not doc.strip():
                raise TypeError(
                    f"Class '{name}' must define a non-empty docstring.")
            if "validate" not in namespace:
                raise TypeError(
                    f"Class '{name}' must implement a 'validate(self)' method.")
        return super().__new__(mcls, name, bases, namespace)
```

Explanation:

Metaclasses intercept the low-level class-object creation step. The namespace dict is inspected inside `type.__new__` and the required rules are enforced. If a subclass fails verification, a `TypeError` is raised to abort compilation before import completes.